

ASSESSX — MASTER PROJECT KNOWLEDGE BASE

This document is the persistent source of truth for the AssessX project. Read this before making architectural or implementation decisions. Do not replace this architecture with a simpler generic exam website.

1. PROJECT IDENTITY

Product Name

AssessX

Product Category

AI-powered secure examination, assessment, proctoring and interview platform.

Core Vision

AssessX is designed to become a serious, scalable assessment-security platform rather than a simple online examination website.

The platform combines:

- Secure examinations
- AI-powered multimodal proctoring
- Identity verification
- Suspicious activity detection
- Explainable fraud-risk scoring
- Evidence collection and forensic replay
- Coding assessments
- Live human interviews
- AI-assisted interviews
- Candidate evaluation
- Recruiter/admin dashboards

The long-term goal is to create a platform capable of supporting educational institutions, companies, recruitment teams and certification organizations.

2. CORE PRODUCT PHILOSOPHY

AssessX should follow a defense-in-depth security model.

No single signal should determine whether a candidate cheated.

Examples:

A candidate looking away for two seconds should NOT automatically be considered cheating.

A phone detection event alone should NOT automatically fail a candidate.

Instead, multiple signals should be correlated.

Example:

Phone detected

+ Candidate looks toward phone

+ Unknown voice detected

+ Browser/application switch

+ Repeated suspicious behavior

→ High-risk event.

The system should produce:

- Events
- Evidence
- Confidence
- Risk scores
- Explanations
- Recommended human review

It should NOT automatically declare a person guilty solely through AI.

Human review must remain possible.

3. PRODUCT FLOW

The main product is a Windows desktop application.

The public website is primarily a landing page and download portal.

Main flow

User visits:

AssessX Website



Reads about the platform



Downloads Windows application



Installs AssessX



Launches AssessX desktop application



Login



Select role:

- Student
- Admin
- Interviewer/Recruiter (future)



Role-specific dashboard

4. PUBLIC WEBSITE

The public website is NOT the primary examination environment.

Its responsibilities:

- Product landing page
- Product explanation
- Features
- Security explanation
- Documentation
- System requirements
- Windows download
- Version information
- Contact/about information

Possible future pages:

- Pricing
- Organization signup
- Developer documentation
- Privacy policy
- Security documentation

The website should be lightweight and professional.

Suggested stack:

- React
- TypeScript
- Vite
- Tailwind CSS

5. WINDOWS DESKTOP APPLICATION

The actual secure examination environment runs as a downloadable Windows desktop application.

Preferred technology:

- Tauri
- React
- TypeScript
- Rust/native Windows capabilities where necessary

The desktop application must NOT simply be treated as a normal website wrapped in an executable.

It should eventually provide native capabilities required for a secure exam environment.

Potential capabilities:

- Secure exam mode
- Camera access
- Microphone access
- Screen capture
- Display detection
- Application/window awareness where technically and legally appropriate
- Permission management
- Exam session control
- Local security checks
- Network status monitoring
- Native Windows integration

IMPORTANT:

Do not claim that a browser or JavaScript application can provide complete operating-system-level security.

High-security features must use appropriate native desktop capabilities.

6. INITIAL USER ROLES

STUDENT

Student can:

- Login
- View assigned exams
- View upcoming exams
- Run system check
- Complete identity verification
- Complete room/environment check
- Enter secure exam mode
- Take examination
- Submit examination
- View allowed results

ADMIN

Admin can:

- Login
- Create exams

- Edit exams
- Create question banks
- Add candidates
- Assign candidates
- Configure exam settings
- Configure proctoring policies
- Schedule exams
- Monitor exams
- Review suspicious events
- Review evidence
- View reports
- Manage organizations

INTERVIEWER / RECRUITER

Future role.

Can:

- Schedule interviews
- Join live interview
- View candidate information
- Use AI interview copilot
- Review interview transcript
- Review candidate evaluation
- Review integrity events

7. INITIAL DEVELOPMENT PRIORITY

DO NOT attempt to implement the complete platform at once.

Development must happen incrementally.

Phase 1 — Foundation

Build:

- Repository structure
- Public website
- Windows desktop application shell
- Backend shell
- PostgreSQL connection
- Basic authentication architecture
- Student role

- Admin role
- Basic dashboards
- Development environment
- Docker setup
- Environment configuration

Success criteria:

Website works.

↓

User downloads application.

↓

Application installs.

↓

Application launches.

↓

Login works.

↓

Student/Admin dashboards work.

Do NOT implement complex AI yet.

8. PHASE 2 — EXAM ENGINE

Implement:

- Exam creation
- Question bank
- MCQ questions
- Multiple question types
- Exam timing
- Question navigation
- Randomization
- Candidate assignment

- Exam submission
- Automatic evaluation
- Result storage

Future question types:

- MCQ
 - Multiple select
 - True/false
 - Short answer
 - Long answer
 - Coding
 - Aptitude
-

9. PHASE 3 — SYSTEM VERIFICATION

Before starting an exam:

Student enters a system-check workflow.

Check:

- Camera
- Microphone
- Screen capture
- Internet connection
- Required permissions
- Display configuration
- Application version
- Audio/video availability

Then:

Identity verification.

Then:

Exam instructions.

Then:

Secure exam mode.

10. IDENTITY VERIFICATION

Potential architecture:

Government/college/company ID

+ Live camera capture

+ Face detection

+ Face embedding comparison

+ Liveness verification

Possible events:

- FACE_VERIFIED
- FACE_MISMATCH
- FACE_MISSING
- LIVENESS_FAILED
- MULTIPLE_FACES

Identity verification should be treated as a security signal.

Do not claim perfect recognition.

11. ROOM / ENVIRONMENT CHECK

Before high-security exams, optionally require a camera/environment scan.

Detect potential:

- Additional person
- Phone
- Tablet
- Books
- Notes
- Additional monitor
- Unauthorized devices
- Other prohibited objects

The system should generate events, not immediate guilt decisions.

Example:

ROOM_SCAN_COMPLETE

PHONE_DETECTED

ADDITIONAL_PERSON_DETECTED

ADDITIONAL_MONITOR_DETECTED

12. AI PROCTORING ENGINE

The proctoring system is multimodal.

Inputs:

1. Camera/video
2. Audio
3. Screen
4. Browser/application activity where technically possible
5. Exam interaction behavior
6. Identity signals

Potential technologies:

- OpenCV
 - MediaPipe
 - YOLO
 - PyTorch
 - Speech/audio models
 - Optional LLM reasoning
-

13. COMPUTER VISION

YOLO

Use for object detection.

Potential classes:

- Person
- Cellphone
- Laptop
- Tablet

- Book
- Monitor
- Headphones
- Calculator
- Other relevant objects

Do not invent unsupported classes without appropriate training/model support.

MediaPipe

Potential uses:

- Face landmarks
- Pose
- Head orientation
- Eye/gaze-related features

OpenCV

Use for:

- Frame processing
- Image processing
- Camera pipeline
- CV utilities

PyTorch

Use for:

- Custom ML models
- Training
- Inference
- Future anomaly detection models

14. GAZE AND HEAD MOVEMENT

Track features such as:

- Head yaw
- Head pitch
- Head roll
- Eye/gaze direction
- Duration of looking away

- Frequency of directional deviations
- Repeated behavioral patterns

Do NOT automatically classify every glance as cheating.

The risk engine must consider duration, frequency and contextual correlation.

15. AUDIO INTELLIGENCE

Analyze microphone input for:

- Voice activity
- Multiple speakers
- Unknown speakers
- Whispering-like activity
- Conversation
- Unusual background speech

Possible events:

- VOICE_DETECTED
- MULTIPLE_SPEAKERS
- UNKNOWN_SPEAKER
- SUSPICIOUS_AUDIO

Audio processing must consider false positives from:

- Family members
 - Hostel/room noise
 - Traffic
 - Fans
 - Background conversations
 - Accessibility needs
-

16. SCREEN MONITORING

Where the security mode permits and user consent/legal requirements are satisfied, monitor screen activity.

Potential events:

- SCREEN_CAPTURE_STARTED
- SCREEN_CAPTURE_STOPPED

- DISPLAY_CHANGED
- WINDOW_CHANGED
- UNAUTHORIZED_APPLICATION
- SUSPICIOUS_BROWSER_ACTIVITY

Potential prohibited applications/sites can include:

- Search engines
- AI assistants
- Messaging applications
- Remote desktop software
- Screen sharing applications

However, do NOT rely exclusively on a static blacklist.

The system should generate evidence and risk signals.

17. SECURE EXAM MODE

The Windows client should eventually provide a high-security exam mode.

Potential protections:

- Prevent normal navigation
- Restrict exam escape
- Detect attempts to switch applications
- Detect display changes
- Restrict copy/paste where appropriate
- Disable or restrict right-click where appropriate
- Detect suspicious system state
- Monitor relevant process/application state
- Detect exam client tampering
- Maintain session integrity

IMPORTANT:

Do not falsely claim that every operating-system bypass can be prevented.

Document technical limitations.

18. EVENT-BASED PROCTORING

The proctoring engine should produce structured events.

Example:

```
{
  "event_type": "PHONE_DETECTED",
  "timestamp": "...",
  "confidence": 0.94,
  "source": "yolo",
  "severity": "medium"
}
```

Possible event types:

- FACE_MISSING
- FACE_MISMATCH
- MULTIPLE_FACES
- LIVENESS_FAILED
- PHONE_DETECTED
- TABLET_DETECTED
- EXTRA_PERSON
- EXTRA_MONITOR
- GAZE_DEVIATION
- HEAD_POSE_DEVIATION
- UNKNOWN_VOICE
- MULTIPLE_SPEAKERS
- SCREEN_CHANGED
- APPLICATION_CHANGED
- TAB_SWITCH
- COPY_ATTEMPT
- PASTE_ATTEMPT
- NETWORK_CHANGE
- DISPLAY_CHANGE
- EXAM_CLIENT_VIOLATION

Events must be timestamped.

19. RISK ENGINE

This is one of the most important components of AssessX.

The risk engine correlates multiple events.

Example:

GAZE_DEVIATION

+ PHONE_DETECTED

+ UNKNOWN_VOICE

+ APPLICATION_SWITCH

→ higher risk than any individual event.

Initial risk scoring can be rule-based.

Future versions can use:

- Statistical models
- Gradient boosting
- Anomaly detection
- ML-based risk scoring

Do not start with an unnecessarily complex ML model.

Start with an explainable deterministic system.

20. RISK LEVELS

Suggested levels:

LOW

Normal behavior or weak isolated anomaly.

MEDIUM

Repeated suspicious behavior.

HIGH

Multiple correlated suspicious events.

CRITICAL

Strong evidence requiring immediate review.

Example:

0–25: Normal

26–50: Low

51–75: Medium

76–100: High/Critical depending on policy

These thresholds must eventually be configurable by the organization.

21. EXPLAINABLE ALERTS

Never show only:

"CHEATING DETECTED"

Instead:

HIGH RISK EVENT

Time: 14:32:18

Evidence:

- Phone detected
- Candidate gaze directed toward phone
- Unknown voice detected
- Application change detected

Confidence:

87%

Recommended action:

Review the session segment between 14:30 and 14:34.

This is a core design principle.

22. EVIDENCE ENGINE

Store structured evidence associated with suspicious events.

Possible evidence:

- Timestamp
- Screenshot/frame
- Short video segment
- Audio segment where legally appropriate
- Screen event
- Detection metadata
- Model confidence
- Event sequence

Avoid unnecessary continuous storage if event-based evidence is sufficient.

Data retention should be configurable.

23. FORENSIC REPLAY

Admin/proctor should be able to review an examination timeline.

Example:

10:02 — Identity verified

10:10 — Exam started

10:19 — Gaze deviation

10:27 — Phone detected

10:27 — Unknown voice

10:31 — Application switch

10:45 — Normal

Clicking an event should take the reviewer directly to the relevant evidence.

24. HUMAN REVIEW

AI should assist human reviewers.

Workflow:

AI detects event



Creates evidence



Calculates risk



Human reviewer reviews



Confirm / Dismiss / Escalate

This is especially important for high-stakes exams.

25. AI PROCTOR COPILOT

Future feature.

Admin/proctor dashboard:

Candidates online: 500

Normal: 421

Low risk: 52

Medium risk: 21

High risk: 6

Critical: 0

The AI prioritizes candidates for human review.

26. COLLUSION DETECTION

Future advanced feature.

Compare candidates using:

- Answer similarity
- Submission timing
- Question navigation patterns
- Device metadata where permitted
- Network metadata where permitted
- Identical unusual wrong answers
- Behavioral synchronization

Potential output:

SUSPICIOUS CANDIDATE CLUSTER

Candidate 14 Candidate 28 Candidate 51

Reason:

Highly similar unusual answers submitted within a short time window.

Do not use IP address alone to accuse candidates.

27. CODING EXAM

Future major module.

Provide:

- Code editor
- Multiple languages
- Compilation
- Test cases
- Runtime limits
- Memory limits
- Hidden test cases
- Submission history

Languages may include:

- C

- C++
- Java
- Python
- JavaScript
- Go

Code must execute in isolated sandboxes.

NEVER execute untrusted candidate code directly on the main API server.

28. CODE SIMILARITY

Compare candidate submissions using:

- AST
- Structural similarity
- Token similarity
- Embeddings where appropriate

Goal:

Detect likely collaboration/plagiarism.

Do not claim that similarity automatically proves cheating.

29. AI INTERVIEW SYSTEM

AssessX will eventually support two interview modes.

AI-ONLY

AI interviewer conducts the interview.

HUMAN LIVE INTERVIEW

Candidate and interviewer communicate through live video/audio.

30. LIVE HUMAN INTERVIEW

The interview should support:

- Candidate video
- Interviewer video
- Microphone
- Camera
- Screen sharing
- Chat
- Session timer
- Interview notes
- Optional recording
- AI analysis
- Proctoring

Communication technology:

- WebRTC
- WebSockets where appropriate
- Jitsi or LiveKit may be evaluated

WebRTC is the communication layer, NOT the security layer.

Proctoring remains independent.

31. LIVE INTERVIEW ARCHITECTURE

Candidate

↓

WebRTC/Jitsi/LiveKit

↓

Interviewer

Meanwhile:

Candidate camera

+ Microphone

+ Screen

+ Relevant session events

↓

AI Proctoring Engine

↓

Risk Engine

The interviewer can receive high-priority integrity alerts.

32. AI INTERVIEW COPILOT

The AI should assist the interviewer.

Example:

Candidate answers a C++ question.

AI analyzes:

Technical accuracy: 84%

Depth: 72%

Communication: 91%

Missing concept:

Collision resolution

Suggested follow-up:

"What happens when two keys produce the same hash?"

The AI should assist rather than automatically control every human interview.

33. ADAPTIVE AI INTERVIEW

Interview flow:

Question



Candidate answer



LLM evaluates answer



Generate appropriate follow-up



Adjust difficulty



Continue

Topics can include:

- DSA
 - OOP
 - OS
 - DBMS
 - Computer Networks
 - System Design
 - Machine Learning
 - Deep Learning
 - NLP
 - Computer Vision
 - LLMs
 - Behavioral questions
-

34. INTERVIEW SCORECARD

Possible metrics:

- Technical knowledge
- Problem solving
- Coding
- Communication
- Answer relevance
- Reasoning
- Behavioral responses
- Overall interview performance
- Integrity risk

Do not infer candidate quality from protected or irrelevant characteristics.

Avoid simplistic facial-emotion-based hiring decisions.

35. BACKEND ARCHITECTURE

Primary backend:

Python + FastAPI

Database:

PostgreSQL

Cache/event infrastructure:

Redis

Potential future message broker:

Kafka / RabbitMQ / Redis Streams depending on scale requirements.

Vector database:

Qdrant if RAG/semantic retrieval is required.

36. SUGGESTED SERVICE STRUCTURE

```
services/  
  
api/  
  
proctor/  
  
interview/  
  
risk-engine/  
  
evaluation/  
  
auth/  
  
media/
```

Future services may be split into independent deployable services when scale requires it.

Do not create microservices unnecessarily during the first phase.

Start modular.

Extract services when there is a clear reason.

37. FRONTEND ARCHITECTURE

React + TypeScript.

Suggested:

```
apps/  
  
web/  
  
desktop/  
  
admin/
```

Shared components can live under:

```
packages/
```

```
ui/
```

```
types/
```

```
config/
```

Use strong typing.

Avoid duplicated interfaces.

38. DATABASE

Primary database:

PostgreSQL.

Core entities:

- User
- Organization
- Role
- Exam
- Question
- QuestionOption
- ExamQuestion
- CandidateExam
- ExamAttempt
- Answer
- Submission
- ProctoringSession
- ProctoringEvent
- Evidence
- RiskScore
- Interview
- InterviewParticipant
- InterviewEvent
- InterviewEvaluation
- Report
- AuditLog

Potential future:

- Device
- Session
- QuestionBank
- CodeSubmission
- CollusionCluster

39. MULTI-TENANCY

The platform should eventually support multiple organizations.

Example:

Organization A

- Admins
- Exams
- Candidates
- Reports

Organization B

- Admins
- Exams
- Candidates
- Reports

Tenant isolation must be designed from the beginning.

Do not allow one organization to access another organization's data.

40. AUTHENTICATION

Use secure authentication.

Potential architecture:

- Access tokens
- Refresh tokens
- Secure cookies where applicable
- Role-based authorization
- Organization-level authorization

Roles:

STUDENT

ADMIN

INTERVIEWER

PROCTOR

SUPER_ADMIN (future)

Never store passwords in plaintext.

Never hardcode secrets.

Use environment variables/secrets management.

41. SECURITY PRINCIPLES

Security is a first-class requirement.

Follow:

- Least privilege
- Strong authentication
- Authorization on every sensitive API
- Input validation
- Rate limiting
- Secure headers
- Audit logging
- Encryption in transit
- Encryption at rest where appropriate
- Secret management
- Dependency scanning
- Container security
- Secure file uploads
- Sandbox untrusted code
- Session integrity
- Tamper detection

Never put API keys into Git.

42. PRIVACY

AssessX handles sensitive information.

Therefore:

- Obtain appropriate consent.
- Clearly communicate what is collected.
- Minimize collected data.
- Define retention periods.
- Allow organization-specific retention policies.
- Protect stored evidence.
- Provide appropriate deletion mechanisms.
- Avoid unnecessary biometric storage.
- Separate identification data from analytics where practical.

Compliance requirements must be reviewed for the deployment jurisdiction and use case.

43. FALSE POSITIVES

False positives are a major concern.

Examples:

A student looks away because:

- They are thinking.
- They have a disability.
- They are reading something on screen.
- They are adjusting posture.

A background sound may come from:

- Family
- Hostel
- Traffic
- Construction
- Fan

A second person may briefly enter the room.

Therefore:

Single weak event $\neq$ cheating.

The system should prioritize repeated/correlated evidence.

44. AI MODEL RESPONSIBILITIES

LLMs should be used for:

- Interview conversation
- Question generation
- Answer analysis
- Report generation
- Evidence summarization
- Contextual reasoning

Computer vision models should be used for:

- Object detection
- Face/landmark analysis
- Pose
- Visual events

Do not send every video frame directly to an LLM.

Architecture should be:

Camera

↓

CV models

↓

Structured events

↓

Risk engine

↓

Optional LLM reasoning

45. REAL-TIME ARCHITECTURE

Do not send every video frame through FastAPI.

Conceptual architecture:

Candidate



WebRTC/media pipeline



AI processing workers



Structured events



Redis/Event Bus



Risk Engine



Live alerts



Database

This allows horizontal scaling.

46. SCALABILITY

Long-term target:

Potentially thousands of concurrent candidates.

Architecture should support:

- Horizontal scaling
- Stateless APIs
- Background workers
- Redis
- Message queues
- GPU workers
- Object storage
- Database indexing
- Connection management
- Observability
- Load balancing

Do not claim a specific concurrency number until it has been load tested.

Measure:

- Concurrent candidates
- Processing latency
- Event throughput
- GPU utilization
- CPU utilization
- Memory
- Network bandwidth
- Database throughput
- False-positive/false-negative rates

47. STORAGE

Use object storage for large media/evidence.

Potential:

AWS S3

Do not store large video files directly in PostgreSQL.

PostgreSQL stores:

- Metadata
- References
- Events

- Scores
- Reports

Object storage stores:

- Video
 - Audio
 - Screenshots
 - Evidence clips
-

48. DEPLOYMENT

Development:

Docker Compose

Production:

Docker

AWS

Potential architecture:

Load Balancer

↓

API instances

↓

Redis

↓

PostgreSQL

↓

Worker fleet

↓

GPU workers

↓

Object storage

Production deployment should be introduced after local architecture is stable.

49. OBSERVABILITY

Eventually include:

- Structured logs
- Metrics
- Error tracking
- Request tracing
- Model inference latency
- Queue latency
- GPU utilization
- Proctoring event counts
- WebRTC connection quality

Potential technologies:

- Prometheus
 - Grafana
 - OpenTelemetry
-

50. TESTING

Every major feature requires tests.

Backend:

- Unit tests
- Integration tests
- API tests

Frontend:

- Component tests
- End-to-end tests

Desktop:

- Installation tests
- Permission tests
- Camera tests
- Screen capture tests
- Security-mode tests

AI:

- Model accuracy
- Precision
- Recall
- False positives
- False negatives
- Latency

Load:

- Concurrent sessions
- Event throughput
- Database load
- Worker load

51. GIT WORKFLOW

Use meaningful commits.

Examples:

```
feat: add student authentication

feat: add exam creation API

feat: add secure exam system check

feat: add phone detection event

feat: add proctoring risk engine

fix: resolve exam timer synchronization

docs: add system architecture
```

```
test: add exam submission tests
```

Do not create meaningless commits just to increase commit count.

52. CLAUDE IMPLEMENTATION RULES

Claude is acting as a senior software engineer and architect.

Before implementing a major feature:

1. Inspect the existing repository.
2. Understand current architecture.
3. Identify dependencies.
4. Identify affected modules.
5. Explain the implementation plan.
6. Implement only the requested phase.
7. Run tests.
8. Run the application where appropriate.
9. Fix discovered errors.
10. Update documentation.

Do not rewrite working systems unnecessarily.

Do not introduce new frameworks without a reason.

Do not install large dependencies without evaluating whether they are necessary.

Do not create duplicate services.

Do not create duplicate models/interfaces.

Do not hardcode credentials.

Do not silently change architecture.

53. CLAUDE DEVELOPMENT RULE

Never respond to a large feature request by implementing the entire platform.

Break requests into:

- Architecture
- Backend
- Frontend
- AI
- Testing
- Integration

Implement incrementally.

If a feature conflicts with the architecture, explain the conflict before changing architecture.

54. CURRENT FIRST MILESTONE

The first actual milestone is:

"Download → Install → Login → Dashboard"

Requirements:

Public website

↓

Windows download

↓

Windows installer

↓

AssessX desktop application

↓

Login

↓

Student/Admin role selection

↓

Student dashboard

OR

Admin dashboard

No advanced AI is required for this milestone.

55. CURRENT TECHNOLOGY BASELINE

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS

Desktop

- Tauri
- React
- TypeScript
- Rust/native Windows APIs where required

Backend

- Python
- FastAPI
- Pydantic
- SQLAlchemy
- PostgreSQL
- Redis

AI

- OpenCV
- MediaPipe
- YOLO
- PyTorch
- LLM API

Realtime

- WebRTC
- WebSockets
- Jitsi or LiveKit may be evaluated

Infrastructure

- Docker
- Docker Compose
- AWS eventually
- GitHub
- GitHub Actions

Optional

- Qdrant
- Prometheus
- Grafana
- OpenTelemetry

56. PRODUCT DEVELOPMENT PHILOSOPHY

Build a real product, not a demo full of fake buttons.

Every major UI feature should eventually connect to real backend functionality.

Avoid:

- Fake analytics
- Fake AI scores
- Fake proctoring events
- Hardcoded candidate lists
- Pretend security
- Placeholder "AI detected cheating" logic

During early development, mock data is acceptable only when explicitly marked as mock and isolated so it can later be replaced.

57. UI/UX DIRECTION

AssessX should look like a professional enterprise security/assessment platform.

Visual style:

- Modern
- Clean
- Professional
- Security-oriented
- Minimal unnecessary decoration
- Strong information hierarchy
- Responsive dashboards

Important dashboards:

- Student dashboard
- Admin dashboard
- Live proctoring dashboard
- Candidate investigation page
- Interview room
- Interview copilot
- Analytics
- Evidence timeline

Avoid generic "AI glowing" designs everywhere.

58. KEY DIFFERENTIATOR

AssessX should NOT market itself merely as:

"Webcam detects cheating."

Its core differentiator is:

Multimodal Assessment Integrity

The platform correlates:

Video

+

Audio

+

Screen

+

Identity

+

Device/session signals

+

Exam interaction behavior

+

Answer patterns

↓

Explainable Risk Engine

↓

Evidence

↓

Human Review

59. LONG-TERM FEATURE ROADMAP

Level 1

Core examination

- Authentication
- Question bank
- Student/Admin
- Results

Level 2

Secure exam client

- Camera
- Microphone
- Screen
- Identity
- System checks

Level 3

AI proctoring

- Face
- Gaze
- Pose
- YOLO
- Audio
- Screen events

Level 4

Risk intelligence

- Event correlation
- Risk score
- Evidence
- Forensic replay

Level 5

Advanced integrity

- Collusion detection
- Code similarity
- Question leak monitoring

Level 6

Live interviews

- WebRTC
- Human interviewer
- Screen sharing
- Interview recording/evidence

Level 7

AI interviewer

- Adaptive questions
- Technical evaluation
- Behavioral evaluation
- Interview copilot

Level 8

Enterprise scale

- Multi-tenancy
 - AWS
 - Horizontal scaling
 - Observability
 - Advanced security
-

60. FINAL RULE

The goal is not to create the largest amount of code.

The goal is to create a coherent, secure, testable and scalable assessment platform.

Prioritize:

1. Correct architecture
2. Security
3. Reliability
4. Explainability
5. Privacy
6. Testing
7. Scalability
8. User experience
9. AI capabilities

Always preserve the long-term architecture when implementing individual features.

END OF ASSESSX MASTER KNOWLEDGE BASE